

Ejemplo de métodos JAVA

Vamos a generar un ejemplo de uso de una clase con métodos estáticos y métodos de instancia. Necesito que en el main de la aplicación se vea la interacción o paso de mensaje entre objetos. Vamos a codificarlo en JAVA

Una buena forma de mostrar la diferencia entre **métodos de instancia** y **métodos estáticos** es utilizar tres clases:

- **Persona**: representa un objeto. Tiene métodos de instancia (`saludar()` , `presentarse()`) y un método estático (`mostrarCantidadPersonas()`).
- **Mensaje**: representa un objeto que envía mensajes entre personas (paso de mensajes entre objetos).
- **App**: contiene el `main` y muestra claramente la interacción.

La idea es que en el `main` se vea:

1. Creación de objetos.
2. Invocación de métodos estáticos.
3. Invocación de métodos de instancia.
4. Paso de mensajes entre objetos (`persona1` le pide a `mensaje` que envíe un mensaje a `persona2`).

Clase Persona.java

</> Java

```
package ejemplo;

public class Persona {

    private String nombre;
    private int edad;

    // Atributo estático (compartido por todas las personas)
    private static int cantidadPersonas = 0;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
        cantidadPersonas++;
    }
}
```

```
// Método de instancia
public void saludar() {
    System.out.println("Hola, soy " + nombre);
}

// Método de instancia
public void presentarse() {
    System.out.println("Me llamo " + nombre + " y tengo " + edad + " años.");
}

// Método de instancia
public String getNombre() {
    return nombre;
}

// Método estático
public static void mostrarCantidadPersonas() {
    System.out.println("Cantidad de personas creadas: " + cantidadPersonas);
}
}
```

Clase Mensaje.java

</> Java

```
package ejemplo;

public class Mensaje {

    // Método de instancia
    public void enviarMensaje(Persona emisor, Persona receptor, String texto) {

        System.out.println("-----");
        System.out.println(emisor.getNombre() + " envía un mensaje a " + receptor.getNombre());
        System.out.println("Mensaje: " + texto);
        System.out.println(receptor.getNombre() + " recibió el mensaje.");
        System.out.println("-----");
    }

    // Método estático
    public static void mostrarEncabezado() {
        System.out.println("=== SISTEMA DE MENSAJES ===");
    }
}
```

Clase App.java

</> Java

```
package ejemplo;

public class App {

    public static void main(String[] args) {

        // =====
        // MÉTODOS ESTÁTICOS
        // =====

        Mensaje.mostrarEncabezado();

        Persona.mostrarCantidadPersonas();

        System.out.println();

        // =====
        // CREACIÓN DE OBJETOS
        // =====

        Persona juan = new Persona("Juan", 25);
        Persona maria = new Persona("María", 22);

        Mensaje servicioMensajes = new Mensaje();

        System.out.println();

        Persona.mostrarCantidadPersonas();

        System.out.println();

        // =====
        // MÉTODOS DE INSTANCIA
        // =====

        juan.saludar();
        maria.saludar();

        System.out.println();

        juan.presentarse();
        maria.presentarse();

        System.out.println();
```

```
// =====  
// PASO DE MENSAJES ENTRE OBJETOS  
// =====  
  
servicioMensajes.enviarMensaje(  
    juan,  
    maria,  
    "Hola María, ¿cómo estás?"  
);  
  
servicioMensajes.enviarMensaje(  
    maria,  
    juan,  
    "Muy bien Juan. ¡Gracias!"  
);  
  
}
```

Salida esperada

=== SISTEMA DE MENSAJES ===

Cantidad de personas creadas: 0

Cantidad de personas creadas: 2

Hola, soy Juan

Hola, soy María

Me llamo Juan y tengo 25 años.

Me llamo María y tengo 22 años.

Juan envía un mensaje a María

Mensaje: Hola María, ¿cómo estás?

María recibió el mensaje.

María envía un mensaje a Juan

Mensaje: Muy bien Juan. ¡Gracias!

Juan recibió el mensaje.

¿Dónde se observa cada concepto?

Métodos estáticos

No necesitan un objeto para ejecutarse.

</> Java

```
Mensaje.mostrarEncabezado();  
  
Persona.mostrarCantidadPersonas();
```

Se invocan utilizando directamente el nombre de la clase.

Métodos de instancia

Necesitan un objeto previamente creado.

</> Java

```
juan.saludar();  
  
maria.presentarse();
```

Aquí el mensaje se envía al objeto (`juan` o `maria`).

Paso de mensajes entre objetos (concepto fundamental de POO)

El objeto `App` le envía un mensaje al objeto `servicioMensajes` :

</> Java

```
servicioMensajes.enviarMensaje(  
    juan,  
    maria,  
    "Hola María, ¿cómo estás?"  
);
```

Luego, el objeto `servicioMensajes` trabaja con **otros dos objetos** (`juan` y `maria`) para completar la operación.

El flujo de mensajes puede representarse así:

```
main()  
|  
▼
```

```
servicioMensajes.enviarMensaje(juan, maria, texto)
    |
    |→ juan.getNombre()
    |
    |→ maria.getNombre()
```

Este ejemplo resulta didáctico porque en un mismo programa se distinguen claramente:

- **Métodos estáticos** (pertenecen a la clase).
- **Métodos de instancia** (pertenecen a cada objeto).
- **Creación de objetos** mediante `new`.
- **Paso de mensajes entre objetos**, mostrando cómo un objeto colabora con otros para realizar una tarea, uno de los principios centrales de la Programación Orientada a Objetos.



el paso de mensajes lo quería implementar mas sencillo...
por ejemplo el objeto Profesor le coloca una nota al
Alumno y el Alumno le impacta en su atributo nota.

Sí, ese ejemplo es mucho más representativo del concepto de **paso de mensajes** en POO porque un objeto modifica el estado de otro a través de un método. Además, es un caso muy cercano a los ejemplos clásicos que se utilizan en la enseñanza de Objetos.

Yo lo implementaría así.

Alumno.java

</> Java

```
package ejemplo;

public class Alumno {

    private String nombre;
    private double nota;

    public Alumno(String nombre) {
        this.nombre = nombre;
        this.nota = 0;
    }

    // Método de instancia
    public void recibirNota(double nota) {
        this.nota = nota;
    }
}
```

```
    }

    // Método de instancia
    public void mostrarDatos() {
        System.out.println("Alumno: " + nombre);
        System.out.println("Nota: " + nota);
    }

    public String getNombre() {
        return nombre;
    }
}
```

Profesor.java

</> Java

```
package ejemplo;

public class Profesor {

    private String nombre;

    public Profesor(String nombre) {
        this.nombre = nombre;
    }

    // Método de instancia
    public void corregirExamen(Alumno alumno, double nota) {

        System.out.println(nombre + " corrige el examen de "
            + alumno.getNombre());

        alumno.recibirNota(nota);

        System.out.println("La nota fue registrada.");
    }
}
```

App.java

</> Java

```
package ejemplo;

public class App {
```

```
public static void main(String[] args) {  
  
    Alumno alumno = new Alumno("Juan");  
    Profesor profesor = new Profesor("Laura");  
  
    System.out.println("Estado inicial");  
    alumno.mostrarDatos();  
  
    System.out.println();  
  
    profesor.corregirExamen(alumno, 9.5);  
  
    System.out.println();  
  
    System.out.println("Estado final");  
    alumno.mostrarDatos();  
}  
}
```

Salida

Estado inicial

Alumno: Juan

Nota: 0.0

Laura corrige el examen de Juan

La nota fue registrada.

Estado final

Alumno: Juan

Nota: 9.5

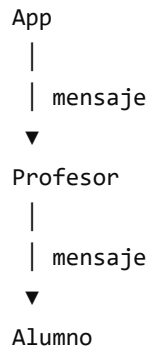
¿Dónde está el paso de mensajes?

En esta línea del `main`:

`</>` Java

```
profesor.corregirExamen(alumno, 9.5);
```

Ocurre lo siguiente:



Es decir:

1. El objeto **App** le envía el mensaje `corregirExamen()` al objeto **Profesor**.
2. El objeto **Profesor** recibe un objeto **Alumno** como parámetro.
3. Dentro del método, el **Profesor** le envía otro mensaje al **Alumno**:

</> Java

```
alumno.recibirNota(nota);
```

4. El **Alumno** modifica su propio atributo `nota`.

Este es el concepto puro de **paso de mensajes**: un objeto no modifica directamente los atributos de otro, sino que le solicita que cambie su estado mediante uno de sus métodos.

Incluso puede hacerse más explícito para los alumnos

</> Java

```
public void corregirExamen(Alumno alumno, double nota) {  
  
    System.out.println("Profesor -> Alumno: recibirNota(" + nota + ")");  
  
    alumno.recibirNota(nota);  
}
```

De esa forma, cuando ejecuten el programa, verán literalmente el "mensaje" que un objeto envía a otro, lo que ayuda mucho a comprender uno de los principios fundamentales de la Programación Orientada a Objetos.

